# Lecture 05.04: Constructor — The init() Method

## 📌 Executive Summary

The role of **init** in instance initialization, difference between **new** and **init**, the self parameter, and object lifecycle.

---

## 🔬 Theoretical Foundations & Principles

### 1. Instance Creation Lifecycle

Object instantiation in Python is a two-phase process:

1. `__new__(cls, *args, **kwargs)` : Allocates raw memory on the heap and returns the new instance.
2. `__init__(self, *args, **kwargs)` : Initializes the attributes of the allocated instance.

$$\text{obj} = \text{Class}(*\text{args}) \implies \text{instance} = \text{Class.__new__}(Class), \quad \text{Class.__init__}(\text{ins}$$

### 2. The `self` Parameter

`self` is an explicit reference to the current instance being manipulated. It is passed automatically by CPython during method invocations.

---

## 💻 Production Implementation & Code

```
class BankAccount:
    def __init__(self, owner: str, initial_balance: float = 0.0):
        self.owner = owner
        self.balance = initial_balance
        print(f"Account created for {self.owner} with ${self.balance:,.2f}")

acc1 = BankAccount("Alice", 1000.0)
acc2 = BankAccount("Bob", 250.0)
```

---

## ⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.